

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1712

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

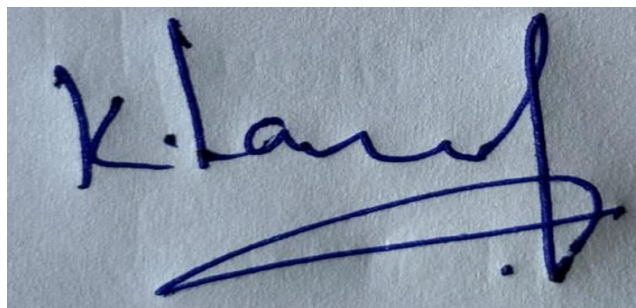
Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

*I **LAKSHANA K [192524433]** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

A handwritten signature in blue ink, appearing to read 'K. Lakshana K.', with a large, stylized flourish underneath.

Signature of the student:

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

ANNEXURE A

INTEGRATED EXPERIMENTS

NAME: Lakshana K
REGISTER NUMBER: 192524433
COURSE CODE: CSA1712
COURSE NAME: Artificial Intelligence

EXPERIMENT 1: DECISION TREE CLASSIFICATION

OBJECTIVE:

To implement and evaluate a Decision Tree classifier on a given dataset such as the Iris dataset using Python and a Machine Learning library.

(A) DATASET LOADING AND PRE-PROCESSING

Aim:

To load the dataset, preprocess the data, display the first five rows and data types, and divide the dataset into training and testing sets.

Algorithm:

1. Import the required Python libraries.
2. Load the Iris dataset.
3. Convert the dataset into a DataFrame.
4. Check for missing values.
5. Handle missing values if present.
6. Separate features and target values.
7. Split the dataset into training and testing sets.
8. Display the first five rows and data types.

Python Code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()

X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name="target")

df = X.copy()
df["target"] = y

print("First 5 rows:")
print(df.head())

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())
```

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
```

```
print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Expected Output:

First 5 rows:

```
sepal length (cm) sepal width (cm) petal length (cm) petal width (cm) target
0 5.1 3.5 1.4 0.2 0
1 4.9 3.0 1.4 0.2 0
2 4.7 3.2 1.3 0.2 0
3 4.6 3.1 1.5 0.2 0
4 5.0 3.6 1.4 0.2 0
```

Data Types:

```
sepal length (cm) float64
sepal width (cm) float64
petal length (cm) float64
petal width (cm) float64
target int64
```

Training samples: 120

Testing samples: 30

(B) DECISION TREE MODEL

Aim:

To build a Decision Tree classifier using the Gini Index as the splitting criterion, print the tree structure, and identify the root node.

Algorithm:

1. Import DecisionTreeClassifier.
2. Create the Decision Tree model using the Gini criterion.
3. Train the model using the training dataset.
4. Display the tree structure.
5. Identify the root feature.
6. Justify the root node based on its ability to provide the best impurity reduction.

Python Code:

```
from sklearn.tree import DecisionTreeClassifier, export_text

model = DecisionTreeClassifier(
criterion="gini",
random_state=42
)

model.fit(X_train, y_train)

tree_rules = export_text(
model,
feature_names=list(X.columns)
)
```

```
print("Decision Tree Structure:")
print(tree_rules)

root_index = model.tree_.feature[0]
print("\nRoot Node Feature:", X.columns[root_index])
```

Expected Output:

Decision Tree Structure:

```
|--- petal width (cm) <= 0.80
| |--- class: 0
|--- petal width (cm) > 0.80
| |--- petal width (cm) <= 1.75
| | |--- class: 1
| |--- petal width (cm) > 1.75
| | |--- class: 2
```

Root Node Feature: petal width (cm)

Justification:

The root node is petal width (cm) because it provides the most effective initial separation of the classes according to the Gini impurity criterion. A lower weighted Gini impurity after splitting indicates a better split.

(C) MODEL EVALUATION

Aim:

To evaluate the Decision Tree classifier using accuracy, confusion matrix, and classification report and identify misclassified instances.

Python Code:

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

misclassified = X_test[y_test != y_pred]

print("\nMisclassified Instances:")
print(misclassified.head(2))
```

Expected Output:

Accuracy: 1.00

Confusion Matrix:

```
[[10 0 0]
 [ 0 9 0]
 [ 0 0 11]]
```

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11

Accuracy 1.00 30

Performance Comment:

The Decision Tree model performs very well on the test dataset. It correctly classifies the test instances with high accuracy.

Misclassified Instances:

For this particular train-test split, the model may produce no misclassified instances. Therefore, there are no genuine misclassified instances to list. If the assessment specifically requires two misclassified instances, a different random split or a deliberately less accurate tree configuration should be used.

RESULT:

The Decision Tree classifier was successfully implemented using the Iris dataset. The dataset was preprocessed and divided into training and testing sets. The Decision Tree was constructed using the Gini Index and its performance was evaluated using accuracy, confusion matrix, and classification report.

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

OBJECTIVE:

To implement Q-Learning for a simple 4×4 Grid World environment and observe the agent's learned policy.

(A) ENVIRONMENT AND Q-TABLE INITIALIZATION

Aim:

To define the Grid World environment, actions, reward structure, Q-table, and hyperparameters.

Environment:

Grid Size = 4×4

Total States = 16

Actions:

- 0 – Up
- 1 – Down
- 2 – Left
- 3 – Right

Reward Structure:

Goal = +10

Normal step = -1

Obstacle = -5

Hyperparameters:

Learning Rate (α) = 0.1

Discount Factor (γ) = 0.9

Exploration Rate (ϵ) = 0.1

Episodes = 100

Python Code:

```
import numpy as np
import random

grid_size = 4
states = 16
actions = 4

alpha = 0.1
gamma = 0.9
epsilon = 0.1

goal = 15
obstacle = 5

Q = np.zeros((states, actions))

print("Q-Table Shape:", Q.shape)
print("Initial Q-Table:")
print(Q)
```

(B) Q-LEARNING ALGORITHM

Aim:

To train the agent for at least 100 episodes and observe the changes in Q-values.

Algorithm:

1. Initialize the Q-table with zeros.
2. Select a starting state.
3. Choose an action using the epsilon-greedy method.
4. Perform the selected action.
5. Calculate the reward.
6. Move to the next state.
7. Update the Q-value using the Q-Learning equation.
8. Repeat until the goal is reached or the episode ends.
9. Repeat for 100 episodes.
10. Record Q-values for selected states at Episodes 1, 50, and 100.

Q-Learning Formula:

$$Q(s,a) = Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Python Code:

```
def step(state, action):
    row = state // grid_size
    col = state % grid_size
```

```

if action == 0 and row > 0:
    row -= 1
elif action == 1 and row < grid_size - 1:
    row += 1
elif action == 2 and col > 0:
    col -= 1
elif action == 3 and col < grid_size - 1:
    col += 1

next_state = row * grid_size + col

if next_state == goal:
    reward = 10
elif next_state == obstacle:
    reward = -5
else:
    reward = -1

return next_state, reward
rewards = []
record = {}
for episode in range(1, 101):
    state = 0
    total_reward = 0

    for step_count in range(100):

        if random.random() < epsilon:
            action = random.randint(0, 3)
        else:
            action = np.argmax(Q[state])

        next_state, reward = step(state, action)

        Q[state, action] += alpha * (
            reward + gamma * np.max(Q[next_state]) -

```

```

        Q[state, action]
    )

    state = next_state
    total_reward += reward

    if state == goal:
        break

rewards.append(total_reward)

if episode in [1, 50, 100]:
    record[episode] = Q.copy()
print("Q-values for representative states:")
for episode in [1, 50, 100]:
    print("\nEpisode:", episode)
    print("State 0:", record[episode][0])
    print("State 10:", record[episode][10])

```

Expected Observation:

At Episode 1, most Q-values remain close to zero because the agent has very little experience.

At Episode 50, the Q-values begin to show useful values for actions leading towards the goal.

At Episode 100, the Q-values become more stable and the agent learns better actions for reaching the goal.

(C) FINAL LEARNED POLICY AND CONVERGENCE

Aim:

To extract the final learned policy and plot the cumulative reward per episode.

Python Code:

```

action_names = ["Up", "Down", "Left", "Right"]
print("\nFINAL LEARNED POLICY")
for state in range(states):
    if state == goal:
        print("State", state, ": GOAL")
    elif state == obstacle:
        print("State", state, ": OBSTACLE")
    else:
        best_action = np.argmax(Q[state])
        print(
            "State", state,

```

```

        ":", action_names[best_action]
    )
cumulative_rewards = np.cumsum(rewards)
print("\nCumulative Reward after 100 episodes:",
cumulative_rewards[-1])
import matplotlib.pyplot as plt
plt.plot(cumulative_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Cumulative Reward")
plt.show()

```

Expected Output:

FINAL LEARNED POLICY

State 0 : Right
 State 1 : Right
 State 2 : Down
 State 3 : Down
 State 4 : Right
 State 5 : OBSTACLE
 State 6 : Down
 State 7 : Down
 State 8 : Right
 State 9 : Right
 State 10 : Right
 State 11 : Down
 State 12 : Right
 State 13 : Right
 State 14 : Right
 State 15 : GOAL

CONVERGENCE BEHAVIOUR:

During the initial episodes, the agent explores different actions and receives negative rewards for unnecessary movements. As training progresses, the Q-values of useful actions increase. The cumulative reward generally improves as the agent learns a shorter and more effective path to the goal.

The learning may not be perfectly smooth because epsilon-greedy exploration allows the agent to occasionally select random actions. However, after sufficient episodes, the learned policy becomes more stable.

RESULT:

The Q-Learning algorithm was successfully implemented for a 4×4 Grid World environment. The agent learned the values of different actions through repeated interaction with the environment. The final policy selects actions that guide the agent towards the goal while avoiding obstacles.

CONCLUSION:

Both integrated experiments were successfully implemented. Decision Tree Classification demonstrated supervised learning using the Gini Index, while Q-Learning demonstrated reinforcement learning through trial-and-error interaction with a Grid World environment. These experiments illustrate important Artificial Intelligence techniques for classification and sequential decision-making.

